

The crew plugin: what it does and why

The combined guide: high-level and technical in one document. Two companion documents split it: *crew-overview* for readers without technical depth and *crew-technical-reference* for the people who run it. Written 20 September 2026 against crew 0.19.92 on main (commit 621d50dc), with the Stop-gate fix (0.19.93) described as it stands in review.

In one paragraph. Crew turns Claude Code from a single assistant into a small, disciplined software team. One agent manages, one investigates, one implements, an independent one from a different AI vendor reviews, and automatic gates stop anything that fails from being merged or released. Everything it knows about a codebase is written down in files that live with the code, so nothing depends on one person's memory or one chat session. It exists because an AI that writes code, reviews its own work, and never loses focus is a fiction; crew makes each of those a separate, checkable step.

1. The problem it solves

Left to itself, an AI coding assistant has four habits that cost real money:

- **It marks its own homework.** The same model that wrote a change is the one saying it is fine.
- **It loses the picture.** Each session starts from nothing. It does not know how the database is shaped, which service calls which, or what broke last month.
- **It wanders.** Asked to fix one thing, it notices five others and fixes those too, and the ticket you asked for arrives with strangers attached.
- **Nothing stops it.** A wrong command against a production database, or a merge with a failing test, goes through unless a human happens to be watching.

Crew answers each habit with a mechanism rather than an instruction: a separate reviewer, a written code map, a scope rule with an enforced write boundary, and gates that block.

2. How it works, in plain terms

Think of it as a very small consultancy inside your editor. You write a ticket. A manager (the PM) decides who should do what and keeps the overall picture.

A developer takes the ticket into a private workspace and does only that job. When it is done, a reviewer from a different company reads the change looking for defects, and reports them in a fixed format. A set of automatic checks runs before anything is merged or promoted, and if one fails, the release stops. Every decision and every lesson is written into files in the repository, so the next session, or the next person, starts where this one left off.

The technical version of the same sentence: file-backed tickets; one implementation session in its own context; an independent QA pass by Codex, Copilot or a fresh Claude context, whichever probes clean; deterministic gates wired into Claude Code's hook system; and a code map, code graph and diagrams kept in version control.

The people, so to speak

Role	What it does	Why it is a separate agent
PM	Reads project state, decides what the crew should do next, dispatches the roles that do it, reports back. Never writes application code itself.	Keeps the whole picture in one place. From 0.19.92 it is mechanically prevented from writing code by the write guard.
Explorer	Answers "where does this live, how does this flow work" and returns a short map, never file contents.	Holds no write or shell tools, so it can be trusted with any question.
Developer	Implements one scoped change and returns a summary. Runs its gates in the foreground. Never reviews its own diff.	Its own context window means the main session's history does not leak into the change.
QA reviewer	A hostile review of a code diff: defect lines or the single word CLEAN.	Independent eyes. Codex (OpenAI) is tried first so the reviewer is a different AI family from the author.
Security	Read-only review before anything touching authentication, secrets, SQL, uploads, infrastructure permissions or CI is merged.	Restricted tools and a separate context.

Smoke author	Writes and repairs the quick regression checks a repository runs on every change.	Repositories with no tests get a harness first, so the gates have something to stand on.
Planner, analyst, researcher	Second opinion on a design; survey of real gaps in an application; answers from outside the repository with sources.	Each works from a brief rather than from the code, so it cannot be led by the implementation.
DBA, docs writer, browser tester, scribe, infrastructure architect	Migration review; documentation from code; Playwright UI tests; the durable record of decisions; cloud topology review before it is built.	Added as a repository grows (the "tiers" below), never by default.

Tiers: the crew grows only when there is evidence it should

Tier	Roles	Added when
0	explorer, qa-reviewer	Always. This is the starting crew.
1	+ security, smoke-author, developer	Security findings reach review; coverage gaps cause regressions; the PM is running work end to end.
2	+ dba, docs-writer, browser-tester, analyst, planner, infrastructure-architect, scribe, researcher	Migrations are routine; a UI regression reached users; designs get reworked after starting; decisions are lost six weeks later; answers are invented instead of looked up.
3	parallel sessions, worktrees, agent teams	Every involved repository has a green smoke harness.

A role earns its place only if it buys an isolated context, a restricted tool set, or genuinely independent eyes. A persona in a prompt buys none of those, so crew does not ship "a business analyst agent" for its own sake. The command `/crew:scale` reads the review metrics and recommends growing, shrinking or staying.

Specialists: 40 more roles you switch on per repository

These are real agent definitions with no tier, because "this repository does SharePoint" is a fact about one checkout, not a stage every project reaches. You onboard one with `/crew:pm onboard <role>` and it justifies itself by the stack, not by metrics.

Group	Specialists
Languages and frameworks	python-pro, node-developer, php-pro, rust-engineer, react-specialist, angular-architect, dotnet-core-expert, dotnet-framework-4.8-expert, sql-pro, graphql-architect, backend-developer, api-designer
Cloud and infrastructure	terraform-engineer, network-engineer, platform-engineer, microservices-architect, legacy-modernizer, database-administrator, git-workflow-manager, workflow-orchestrator, multi-agent-coordinator
Microsoft platform	windows-infra-admin, exchange-online-specialist, sharepoint-developer, power-automate-specialist, powershell-5.1-expert, powershell-7-expert, powershell-security-hardening, ad-security-reviewer
Security and compliance	penetration-tester, compliance-auditor, code-reviewer, architect-reviewer, fintech-engineer, payment-integration
Review and writing	qa-researcher (checks a change's claims about the outside world against live sources), kimi-consult (a second opinion from a third AI family), ai-writing-auditor, skill-author, design-bridge

3. Commands: what you can ask for

Every command is typed as `/crew:<name>` in Claude Code. The right-hand column says what it means for someone who does not read code.

Command	What it does	In plain terms
init	Guided, resumable, phased setup of a repository for crew.	The onboarding wizard. Run once per repository.
onboard	Learns the codebase once and writes a durable, verifiable code map.	Builds the "how this system works" notes so nobody has to re-explain it.
ticket	Scopes a request into a ticket.	Turns "can we make X faster" into a bounded piece of work with acceptance criteria.
work	Picks up a ticket and works it end to end: implement, review, gate.	The daily loop. One ticket in, one reviewed change out.
review	Independent QA review of the current diff by Codex, Copilot or Claude, whichever probes clean first.	A second pair of eyes from a different vendor before anything merges.
verify	Builds or refreshes the verification map from evidence; can run the whole map.	Decides which checks run for which files, and proves they run.
gate	Takes a repository's merge gate down and puts it back, from an export.	The controlled way to bypass protection for an emergency, with a record.
promote	Promotes a build to the next environment with the full post-deploy proof.	Development to QA to production, with evidence at each step.
change	Files, checks and closes a change request in ServiceDesk Plus, Jira or locally.	The change-board paperwork, filled from the ticket and refused while questions are unanswered.

emergency	Declares an incident: stands gates down, opens parallel investigation lanes, records what was skipped.	The break-glass path, with an audit trail of what was bypassed.
debug	Finds the cause of a defect before anyone proposes a fix.	Systematic debugging: reproduce, isolate, prove the cause, then fix.
pm	Talks to the crew's manager: status, assign work, set authority, onboard or offboard roles.	Ask "where are we", or hand it the queue.
plan	An independent design opinion before building.	A second architect who has not seen the code.
survey	Researches the application for real gaps and proposes options with trade-offs.	"What should we improve next, and what does each option cost."
split	Splits an oversized Jira ticket into sub-tickets, with evidence and confirmation.	Breaks a too-big request into deliverable pieces.
docs	Updates the documents a change should touch, and only those.	Keeps the changelog, README and decision records honest.
reference	Generates the API and feature reference from the code, with anchors.	The living reference manual, regenerated rather than hand-maintained.
diagram	Creates or refreshes architecture, process and data-flow diagrams from the actual code.	Pictures that are checked against the code they describe.
runbook	Writes, updates or audits operational runbooks.	The "what to do at 3 a.m." documents, kept current.
roster, model, scale	Lists every role and its backing model; changes and probes the model per role; reviews whether the crew should grow or shrink.	Who is on the team, what powers them, and whether the team is the right size.

config, upgrade	Shows where every setting comes from and guides the machine-wide config; migrates an older crew setup forward.	Settings and upgrades, with nothing changed silently.
jira-sync, sdp-sync, obsidian-sync	Sync a ticket between the local cache and Jira, ServiceDesk Plus or an Obsidian Kanban board.	Your tracker stays the source of truth; crew mirrors it.
handoff	Writes the handoff note for the next session.	Nothing is lost when a session ends or is cleared.

4. Skills: the know-how behind the commands

A skill is a written procedure the agents load when a task matches it. Twenty ship with crew.

Skill	What it covers
crew-setup, crew-pm, crew-scaling	Bootstrapping a repository; managing the crew itself; when to add or remove roles.
crew-verification, crew-lint, crew-terraform	The verification map, test credentials and Playwright; linters and formatters for PowerShell, PHP, Python, Terraform and JavaScript; terraform-docs and tflint.
crew-debugging	Read the error, reproduce it, check recent changes, gather evidence, then propose a fix. Adapted under MIT licence from the superpowers project.
crew-graph, crew-diagrams, crew-docs, crew-runbooks	The code graph; Mermaid diagrams rendered to PNG or SVG; changelog, README, ADRs and generated docs; operational runbooks.

crew-context, crew-memory	Warning near the context limit, handoff notes, resuming after a clear; durable project memory including Obsidian.
crew-providers, crew-cloud, crew-notify	Codex, Copilot and Gemini as external providers; the official AWS and Azure MCP servers; Teams and Telegram notifications.
crew-change, crew-house-style, crew-best-practices, find-skills	The change-request template; document style for anything handed to a human; community practices for Claude Code; discovering skills from the open ecosystem.

5. Safety: the gates and guards

Claude Code lets a plugin register "hooks": scripts that run automatically at fixed moments, whether or not the model agrees. Crew uses them for the checks that must not depend on anyone remembering. A hook that can block a turn ships **off** and is switched on deliberately.

When it runs	What crew does there	What that means
Session start	Briefs the PM with project state and any handoff note; syncs the platform config.	Every session begins knowing where the last one stopped.
Before a shell command	Guards: refuse terraform apply and tofu apply, force-push, admin merge; production database and host patterns once declared.	A wrong command against production is refused, not regretted.
Before a file write (new in 0.19.92)	The role write guard: each role may write only where its own definition allows. The PM's scope is notes, tickets and diagrams.	The manager cannot quietly become the developer.
Before context compaction	Writes the handoff note.	Nothing is lost when the conversation is summarised.

On a notification	Pulses the PM with what changed.	Work landing elsewhere is noticed.
At the end of every turn (the Stop gate)	Runs the verification map for the files that changed; blocks the turn on failure.	The safety net under every change. Section 8 is about fixing it.

Two further gates sit outside hooks: the **merge gate** (Bitbucket or GitHub branch protection, driven by the same map) and the **promotion gate**, which refuses to promote without a row in the promotions log and, when configured, without a change request.

Every guard resolves to the **narrower** of the repository and machine-wide settings, so a cloned repository cannot loosen a machine's policy, and an unreadable setting resolves to the strict value, never to "off".

6. Knowledge: what crew remembers, and where

- **The code map** (`.crew/codemap/`): one prose file per subsystem, every claim marked as derived from a specific line of code or as judgement, each anchored to a commit so staleness is detectable, not guessed.
- **The code graph** (`graphify-out/`): a mechanical graph of every symbol and call, rebuilt by a hook after each commit, queried by the explorer.
- **Diagrams** (`docs/diagrams/`): Mermaid sources with anchors, refreshed only when the code they cite moved.
- **Runbooks, ADRs, changelog, handoff notes**: the durable record, kept by the scribe and the docs commands.
- **Review metrics** (`.crew/metrics.md`): one row per review, which is how `/crew:scale` and the PM's health signal know whether tickets are too large or a role is earning its place.

Everything above lives in the repository. Crew deliberately keeps nothing that only one machine or one chat can see.

7. What changed this week, and what it means

Between 19 and 20 September, crew moved from 0.19.89 to 0.19.92 on main, with 0.19.93 in review. Each change is the result of a root-cause finding, and each was reviewed by Codex across several rounds before merging.

Version	Change	What it means for you
0.19.90	The idle rule, in both directions. Every role that runs a check runs it in the foreground and never ends its turn waiting on a background task, because a finished agent is never woken. The PM learns the other half: an idle signal is not proof a role stopped; check the workspace's file times against the clock and escalate rather than restart.	Three agents in one night had stalled forever "waiting for a notification". That cannot happen now, and the manager will not throw away live work by restarting it.
0.19.91	A behaviour eval suite. Five live tests, each tempting one documented rule, graded by structure rather than by a judge model. A matched pair of runner scripts and a CI workflow that skips visibly without an API key.	The rules are tested against a real model, not just read. The first run caught the PM writing code itself.
0.19.92	The role write guard. A hook that enforces each role's write scope from its own definition. Off by default; the strictest setting is the floor once enabled. Corrupt configuration forces the strict setting. Paths are resolved through symlinks and junctions with each operating system's own ordering.	Prose said the PM must not write code, and it did anyway for four hours. Now the tool refuses. Enable it per repository with <code>guards.roleWrites: block</code> .
also merged	Frontmatter across all 54 agents and every command is parsed as YAML by the marketplace check, after four command files were found to load with no frontmatter at all. All 21 verification rules in this repository are priced.	Silent breakage in the definitions is now a failing check, and the Stop budget can defer instead of running everything.

8. The Stop-gate fix (0.19.93, in review)

The problem in one sentence. The safety net at the end of every turn was taking seven minutes, every turn, on this machine, and people were cancelling it, which is worse than not having it.

What was wrong

The gate decides what to check by comparing the current files against the last commit it fully verified, recorded in a marker file. That marker only advanced when nothing had been deferred. One rule in this repository costs 185 seconds against a 60-second budget, so it was deferred on every turn, the marker never moved, and every turn re-checked 59 files across 13 rules. Separately, a rule

with no declared cost could never be deferred at all, so ten of them ran whole every turn regardless of budget. Two other repositories on the same machine were worse: one had no marker at all and no priced rules, with a 499-second Stop; the other had ten rules that ran a smoke script which, without a flag, fired commands at a live development web server over SSM on every turn, with the target chosen by an ambient environment variable.

What the fix does

Component	Technical	Plain terms
Per-rule verified record	Each rule that ran and passed on the current tree is recorded by content hash; the record advances rule by rule. A chronically over-budget rule, one excluded for reach, one needing a clean tree, or one that skipped is listed as unverified until <code>/crew:verify --all</code> runs it, and reported every turn.	One slow check no longer freezes all the others. Nothing is forgotten; it is listed.
Measure and cache	An unpriced rule is timed on its first run and the measurement cached locally, so it is deferrable next turn. <code>--price</code> writes seconds into the map for the author; it is never reachable from the hook.	Checks price themselves. Unknown cost means "mandatory once", not "mandatory forever".
Reach declarations	Rules declare <code>local</code> , <code>network</code> or <code>host</code> . Only local rules run on Stop. Inspection may only reject: an undeclared rule that runs any script, inline shell or <code>cd</code> is deferred until the map declares it. A parse-only <code>bash -n</code> stays local.	Nothing reaches a live server unattended. Each repository declares its wrapper scripts once; the PM tells it which.
Environment pinning	<code>ENV</code> , <code>AWS_PROFILE</code> , <code>AWS_DEFAULT_REGION</code> , <code>KUBECONFIG</code> and <code>TF_WORKSPACE</code> are unset for every rule unless the map declares them; the gate prints what it pinned.	A leftover variable in a terminal cannot point a check at production.
Exit 77 as SKIP	The standard "environment absent" code is neither pass nor fail, never advances the marker, and deletes any earlier clean fingerprint.	A check that could not run is not counted as passed.
Three PM triggers	Marker missing or stale by N commits; N rules unpriced; N rules with undeclared reach. An unreadable value renders as "unknown" and fires.	The manager tells you what to declare or price instead of you discovering it from a slow hook.

Review so far

Five Codex rounds plus a PowerShell security specialist have reviewed the branch; 31 findings were fixed with a test for each, and a self-check test now runs this repository's real rule map through the classifier. That self-check caught, in turn, `python -m pytest` being read as a wrapper script, the word "curl" in a comment, and the parse-only case. Four rounds of the wrapper scanner each found a new way shell syntax could hide a script, which is why the design was inverted to reject-only in round four. **One fix pass (splitting compound commands on shell operators) is in progress; then one more review round, then merge.** Its effect on live sessions is not yet measured.

What a repository has to do after upgrading

- Declare reach: `local` on rules that run a script, or the PM pulse will name them and they will be deferred until declared.
- Run `/crew:verify --price` once, or let the cache price rules as they run.
- Create the marker on a verified commit only. Marking an unverified commit as verified means the gate will not look at it again.

9. Integrations

Area	What crew connects to
Trackers	Jira and ServiceDesk Plus through MCP; an Obsidian Kanban board; a local file cache always.
Reviewers and models	Codex (OpenAI) as the first-choice reviewer; GitHub Copilot models including Kimi as an alternative family; Gemini as a design partner; per-role model pins with a probe that confirms the model answers before it is trusted.
Cloud	The official AWS and Azure MCP servers, configured per repository with guardrails.
Notifications	Microsoft Teams and Telegram, outbound; two-way options documented.

Memory	Optional Obsidian vault for cross-repository notes; never written to without explicit consent in the session.
--------	---

10. Getting started

1. Install the marketplace and the crew plugin in Claude Code, then `/reload-plugins` in any open session.
2. In a repository, run `/crew:init`. It is phased and resumable; nothing is written without saying so.
3. Build the smoke harness when init asks. Everything downstream stands on it.
4. Run `/crew:onboard` so the crew learns the codebase once and writes it down.
5. Write a ticket with `/crew:ticket`, then `/crew:work`. Read the review. Merge when the gate is green.

11. Glossary

Term	Meaning
Agent, role	A separately-prompted instance of the model with its own tools and context window.
Hook	A script Claude Code runs automatically at a fixed moment, whether or not the model wants it to.
Gate	A check that blocks progress on failure rather than offering advice.
Guard	A hook that refuses a specific dangerous command or write.
Verification map	The tracked file that says which checks run when which files change, and what each costs.

Marker, fingerprint, record	Machine-local files that remember what the Stop gate has already verified, so it does not repeat work.
-----------------------------	--

Reach	Whether a check stays on this machine (local), touches the network, or touches a host.
-------	--

Code map, code graph	The prose and the mechanical descriptions of a codebase, both kept in the repository.
----------------------	---

Sabotage test	Reintroducing a bug on purpose to prove the test that guards against it goes red.
---------------	---

Squash merge	Landing a branch as one commit, so a version bump and its content arrive together.
--------------	--

Provenance: figures are read from the repository at commit 621d50dc (main) and branch crew-stop-gate-record at b6e8c4bb; counts are 28 commands, 54 agents, 20 skills, 20 hook registrations, 51 test modules. Review counts are from `.crew/metrics.md`. Not verified: the Stop-gate fix's effect on live sessions, pending merge.